



LeetCode 1: Two Sum

1. Problem Title & Link

Title: LeetCode 1: Two Sum

Link: <https://leetcode.com/problems/two-sum/>

2. Problem Statement (Short Summary)

Given an array of integers `nums` and an integer `target`.

Return the indices of the two numbers such that:

$\text{nums}[i] + \text{nums}[j] = \text{target}$

Conditions:

$i \neq j$

You may assume:

Exactly one valid answer exists.

Return the indices in any order.

3. Examples (Input → Output)

Example 1

Input:

`nums = [2,7,11,15]`

`target = 9`

Output:

`[0,1]`

Explanation:

`nums[0] = 2`

`nums[1] = 7`

$2 + 7 = 9$

Example 2

Input:

`nums = [3,2,4]`

`target = 6`

Output:

`[1,2]`

Explanation:



$$2 + 4 = 6$$

Example 3

Input:

nums = [3,3]

target = 6

Output:

[0,1]

4. Constraints

$2 \leq \text{nums.length} \leq 10^4$

$-10^9 \leq \text{nums}[i] \leq 10^9$

$-10^9 \leq \text{target} \leq 10^9$

Important:

Exactly one solution exists.

5. Core Concept (Pattern / Topic)**HashMap**

Secondary Topics:

- Array Traversal
- Complement Technique

6. Thought Process (Step-by-Step Explanation)**Brute Force Thinking**

Check every possible pair.

Example:

[2,7,11,15]

Try:

$$2 + 7 = 9 \checkmark$$

Answer found.

Problem

For:

n = 10000



Checking every pair becomes expensive.

Nested loops:

$O(n^2)$

Better Observation

Suppose:

target = 9

Current number:

2

What number do we need?

$9 - 2 = 7$

This required value is called the:

Complement

Key Idea

For every number:

complement = target - current

Check:

Have we already seen complement?

If yes:

Answer found.

Example Walkthrough

Array:

[2,7,11,15]

target = 9

Step 1

Current:

2

Complement:

$9 - 2 = 7$

HashMap:

{}

7 not found.

Store:



2 → index 0

Step 2

Current:

7

Complement:

$9 - 7 = 2$

HashMap:

{2:0}

Found!

Return:

[0,1]

7. Visual / Intuition Diagram (ASCII Diagram)

Example:

```
nums = [2,7,11,15]
```

```
target = 9
```

```
Current = 2
```

```
Need = 7
```

```
Store:
```

```
{2:0}
```

```
Next:
```

```
Current = 7
```

```
Need = 2
```

```
Found in HashMap
```

```
Answer:
```

```
[0,1]
```

```
Visualization:
```

```
Target = 9
```

```
2 + ? = 9
```

```
Need 7
```



7 + ? = 9

Need 2 ✓

8. Pseudocode (Language Independent)

```
create hashmap

for each index i

    complement = target - nums[i]

    if complement exists

        return
        [storedIndex, i]

    store
    nums[i] -> i
```

9. Code Implementation

Python

```
class Solution:
    def twoSum(self, nums, target):

        hashmap = {}

        for i in range(len(nums)):

            complement = target - nums[i]

            if complement in hashmap:
                return [hashmap[complement], i]

            hashmap[nums[i]] = i
```

Java

```
import java.util.*;

class Solution {
```

```
public int[] twoSum(int[] nums, int target) {  
  
    HashMap<Integer,Integer> map =  
        new HashMap<>();  
  
    for(int i = 0; i < nums.length; i++) {  
  
        int complement =  
            target - nums[i];  
  
        if(map.containsKey(complement)) {  
  
            return new int[]{  
                map.get(complement),  
                i  
            };  
        }  
  
        map.put(nums[i], i);  
    }  
  
    return new int[]{};  
}
```

10. Time & Space Complexity

HashMap Solution

Time Complexity: $O(n)$

Reason:

Single traversal.

Space Complexity: $O(n)$

Need HashMap storage.

11. Common Mistakes / Edge Cases

Mistake 1

Using same element twice.

Example:

nums = [3,2,4]



target = 6

Need:

2 + 4

Not:

3 + 3

because only one 3 exists.

Mistake 2

Storing before checking.

Wrong order can fail in some cases.

Example:

[3,3]

Correct:

Check first

Store later

Edge Cases

Duplicate Values

Input:

[3,3]

target = 6

Output:

[0,1]

Negative Numbers

Input:

[-1,-2,-3,-4,-5]

target = -8

Output:

[2,4]

12. Detailed Dry Run

Input:

nums = [2,7,11,15]

target = 9

i	nums[i]	Complement	HashMap	Action
0	2	7	{}	Store 2
1	7	2	{2:0}	Found

Return:

[0,1]

Another Example

Input:

[3,2,4]

target = 6

i	Value	Need	Map
0	3	3	{}
1	2	4	{3:0}
2	4	2	{3:0,2:1}

Found:

[1,2]

13. Common Use Cases (Real-Life / Interview)

Fast Lookup Problems

HashMap allows:

$O(1)$

average lookup.

Pair Matching

Finding:

Pairs

Complements

Relationships

Financial Systems



Example:

Transactions summing to target value.

Database Indexing

Fast retrieval of matching records.

14. Common Traps (Important!)

Trap 1

Using Nested Loops

$O(n^2)$

Works but not optimal.

Trap 2

Sorting the array.

Problem asks for:

Original indices.

Sorting destroys indices.

Trap 3

Not understanding complement.

Core idea:

Need = target - current

Trap 4

Checking current element against itself.

Must use:

Different indices.

15. Builds To (Related LeetCode Problems)

LC 1 - Two Sum



LC 217 - Contains Duplicate



LC 219 - Contains Duplicate II



LC 49 - Group Anagrams





LC 560 - Subarray Sum Equals K



LC 15 - 3Sum

These strengthen HashMap skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Nested Loops	$O(n^2)$	$O(1)$	Poor
Sort + Two Pointers	$O(n \log n)$	$O(n)$	Good
HashMap	$O(n)$	$O(n)$	Best

Approach 1: Nested Loops

for i:

for j:

Simple but slow.

Approach 2: Sort + Two Pointers

Efficient for values.

Problem:

Need original indices.

Requires extra work.

Approach 3: HashMap (Chosen)

Most common interview solution.

17. Why This Solution Works (Short Intuition)

For every number, we immediately compute the value needed to reach the target:

target - current

A HashMap lets us check in constant time whether that required value has already appeared, allowing the solution to run in linear time.

18. Variations / Follow-Up Questions

Follow-Up 1

Count all valid pairs.

Example:

[1,2,3,4,5]



target = 5

Pairs:

(1,4)

(2,3)

Answer:

2

Follow-Up 2

Return values instead of indices.

Follow-Up 3

Solve for:

3Sum

Related:

Cracking the Coding Interview style progression.

Follow-Up 4

Two Sum in a sorted array.

Use:

Two Pointers

instead of HashMap.

Related:

LC 167

Interview Takeaway

This is arguably the most famous HashMap interview problem.

The key pattern is:

Complement Technique

Whenever you see:

Target Sum

Target Difference

Target Relationship

think:

HashMap + Complement

This pattern appears repeatedly in medium and hard interview problems.

